Java Hello World Program :

```
Public class HelloWorld {
    //program begins with a call to main()
    public static void main(String[] args)
    {
        // Prints "Hello" to the Terminal window
        System.out.println("Hello");
    }
}
```

1. Class Definition :-

*) Every Java Program must have atleast one class.

*) If class is public, the filename must ~~the~~ match the class name "HelloWorld.java"

```
Public class HelloWorld {
        // statements
}
```

File name must be : "HelloWorld.java"

If not : You will get compile-time error.

Why?

- Because Java needs clear entry points and strict mapping:
  - One public class = one file
  - Helps JVM and compiler locate classes easily.

What if class is not public?

↳ Can we give any name to file?

"Yes!"

```
class HelloWorld
{
  Public Static void main (String[] args)
  {
    System.out.println("Hello");
  }
}
```

*) Class is not public, then file name can be anything:

"Test.java"

*) No error will come.

Why is this allowed ?

*) Because Non-public classes are not visible outside the package.

*) So, Java does NOT enforce that strict naming here.

*) But :

If class is not public, while running you must use the class name instead of filename.

"Java Test.~~java~~" X

"java HelloWorld" ✓

2. Main Method :

In java, every program must contain a main method as it is the entry point of application.

```
Public static void main (String[] args)
```

Public -> Allows JVM to access methods from anywhere

Static -> Method can run without creating object.

→ Because : In java class cannot execute without an object.

→ Since main method should be inside class, it is obvious that main method cannot able to executed without object of it's class.

→ So, Static allows to run the method without object.

void → It doesn't return any value.

(String[] args) → Accepts command-line arguments.

String[] → array of strings

args → just variable name (can be changed)

So, it is a string array to stores command line inputs.

What is command line Arguments?

when running, in terminal

java Hello , but you can do like this

java Hello 10 20 kishor

Stored as :

args [0] : "10"
args [1] : "20"
args [2] : "Kishor"

as

- All values are stored as Strings
- To the args [ ] .

→ You can convert if need :

"int num = Integer.parseInt(args[0]);"

→ If you give no input : "java Hello" while running

Accessing args[0] gives error.

*) "Args" can be changed :

public static void main (String[] a)

- JVM will pass input to main() using this array.

What if no "String[] args" ?

if

```
public static void main()
{
  =
}
```

- It will compile
- But wont run

*) `javac Hello.java` // compiling , no error

*) `java Hello` // executing , error

" main method not found in class Hello "

Why this Happens ?

*) JVM looks for exact signature :
"Public static void main (String[] args)."

*) If it doesn't find exact method :
- JVM doesn't know where to start
- So program won't run.

3. System.out.println() :-

*) It prints output to the console.
- System → Built-in class from java.lang package.
- out → Static variable (object) inside "System" class

"Public static final PrintStream out ;"
↳ type

why Static ?
- "Out" belongs to class, not object
- So, we can ~~son~~ use it directly.

- println() → Method that prints to console and moves to the next line.

Command line Arguments in Java :-

→ Java command-line argument is an argument, i.e., passed at the time of running the java program.

→ Command-line arguments passed from the console can be received by java program and used as input.

Example :-

→ command → filename → input 1 → input 2

```
java Geeks Hello World
```

=> typed in console or terminal

→ The words "Hello" and "World" are the command-line arguments.

→ JVM will collect these words and will pass these arguments to main method as an array of strings called args.

→ JVM passes these arguments to program inside args[0] and args[1].

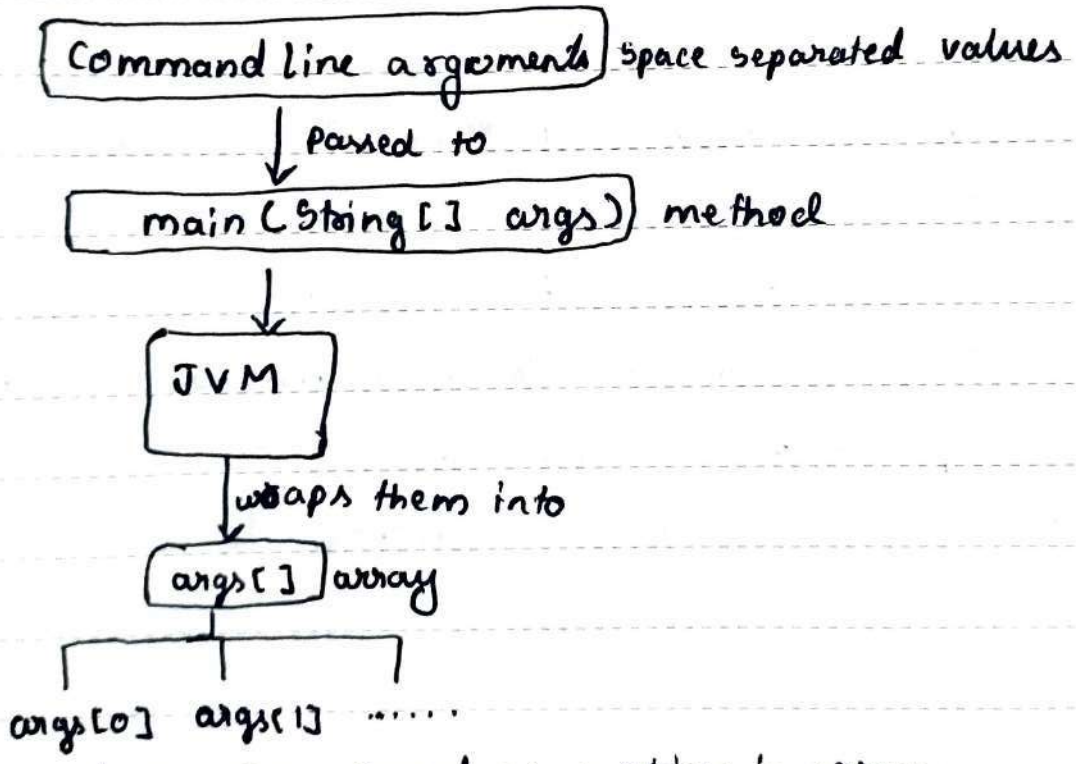
```
class GFG {
    public static void main (String[] args) {
        sout (args[0]);
    }
}
```

Why use Command Line Arguments ?

*) Helps to run programs automatically by giving them the needed information from outside.

*) It is used because it allows us to provide input at runtime without modifying the whole program.

Working of Command Line Arguments :-



values are stored as a strings in array.

*) The no of arguments can be checked using args.length.

How to test this ?

1. Save the program as Hello.java
2. Open Command prompt and compile program " javac Hello.java"
3. run commands line arguments by writing the arguments after "java Hello"

Example:

java Hello Geeks at GeeksforGeeks

4. Press enter , to see output

```
class Hello {
    //Main driver Method
    public static void main (String[] args)
    { if (args.length >0)
        {
            sout("The command line arguments are:");
            for(String val : args)
                sout (val);
        }
        else
            sout ("No command Line arguments found");
    }
}
```